

Student Management System



Software Training Institute, OMR

PROJECT REPORT

ON

Advanced File Handling Application

Using python programming Language

SUBMITTED TO

GOWTHAMI MAM

SUBMITTED BY

I.JAMEEMA BANU

DATA ANALYST

DATE OF SUBMISSION:25.06.2026

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my trainer and mentors for their valuable guidance, support, and encouragement throughout the completion of this Python project.

I am thankful for the opportunity to learn and apply Python programming concepts such as variables, data types, operators, conditional statements, loops, functions, strings, and list operations through this project.

CHAPTER 1: INTRODUCTION TO PYTHON	5
1.1 History and Origin	5
1.2 Key Features of Python	5
1.3 Applications of Python	5
1.4 Advantages of Python	5
1.5 Versions of Python	6
CHAPTER 2: INTRODUCTION TO THE PROJECT	7
2.1 Project Overview	7
2.2 Objectives	7
2.3 Scope	7
2.4 Project Description	7
CHAPTER 3: CONCEPTS USED IN THE PROJECT	9
3.1 Variables	9
3.2 Data Types	9
3.3 Functions	9
3.4 File Handling	9
3.5 OS Module	9
3.6 Date time Module	9
3.7 Login System	9
3.8 Activity Log	9
3.9 Conditional Statements	10
3.10 Loops	10
3.11 Duplicate Detection	10
CHAPTER 4: SOURCE CODE	11
CHAPTER 5: WORKING EXPLANATION	15

5.1 Login and Setup	15
5.2 Create	15
5.3 Read	15
5.4 Append	15
5.5 Rename	15
5.6 Delete	15
5.7 List	16
5.8 Search	16
5.9 Backup	16
5.10 Count	16
5.11 History	16
5.12 Extensions	16
5.13 Duplicate Detection	16
5.14 Activity Log	16
5.15 Flow Chart	17
CHAPTER 6: OUTPUT	19
CONCLUSION	25
BIBLIOGRAPHY / REFERENCES	26

CHAPTER 1: INTRODUCTION TO PYTHON

1.1 History and Origin

Python is a popular high-level programming language created by Guido van Rossum. He started developing Python in 1989, and the first version was released in 1991. Python was designed to make programming simple, clear, and easy to understand. The name "Python" was inspired by the British comedy show Monty Python's Flying Circus. Python is an open-source interpreted programming language that supports variables, data types, functions, loops, and object-oriented programming. Today, Python is widely used in web development, data analysis, artificial intelligence, machine learning, automation, and software development.

1.2 Key Features of Python

- Easy to learn and use
- Simple and readable syntax
- High-level programming language
- Interpreted language
- Open-source and free
- Dynamically typed
- Object-oriented
- Portable and cross-platform
- Large standard library
- Supports multiple programming paradigms
- Large community support

1.3 Applications of Python

Python is used in many areas including Web Development, Data Analysis, Artificial Intelligence (AI), Machine Learning (ML), Automation, Scientific Computing, Data Visualization, Game Development, Desktop Application Development, Cybersecurity, and the Internet of Things (IOT)

1.4 Advantages of Python

- Easy for beginners to learn
- Reduces development time
- Free and open-source
- Large number of libraries and frameworks
- Works on multiple operating systems
- Supports rapid application development
- Strong community support
- Useful for many different types of projects

1.5 Versions of Python

Version	Year	Description
Python 1.0	1994	First major stable release
Python 2.0	2000	Introduced several important improvements
Python 3.0	2008	Major update with significant changes
Python 3.x	2008- present	Continuously improved with new features and updates

CHAPTER 2: INTRODUCTION TO THE PROJECT

2.1 Project Overview

This mini project, the Student Management System, is a Python console application developed to manage and maintain student information using basic Python programming concepts. The system provides a simple and organized way to store, view, search, update, and manage student records.

The application allows the user to enter student details such as student ID, name, age, gender, course, and marks. The user can add new student records, display existing records, search for a particular student, update student information, and delete records when required. The student data is stored in a file so that the information can be maintained and accessed whenever needed.

2.2 Objectives

- To use functions to organize different operations of the application.
- To develop a simple and user-friendly Student Management System using Python.
- To store and manage student information using file handling.
- To provide options to add, view, search, update, and delete student records.
- To implement file handling operations such as read, write, and append.

operations of the application.

- To apply conditional statements, loops, and data structures in a practical project.
- To maintain student records in an organized and efficient manner.

2.3 Scope

The scope of this project is limited to a single-user, file-based console application. It does not use a database or graphical user interface. The system focuses on managing basic student information such as student ID, name, age, gender, course, and marks.

The project demonstrates important Python concepts including file handling, functions, conditional statements, loops, lists, dictionaries, and exception handling. The system can be extended in the future by adding a graphical user interface, database connectivity, multiple-user access, and additional student management features.

2.4 Project Description

The program is organized as a set of functions, with each function responsible for a specific operation in the system. The main functions include `add_Student()` for adding a new student record, `view_students()` for displaying all student records, `search_student()` for finding a particular student, `update_student()` for modifying existing student information, and `delete_student()` for removing a student record.

The system uses file handling to store student information permanently. The main menu allows the user to select the required operation. Input validation and basic exception handling are used to prevent invalid data and errors. The complete program combines different Python programming concepts to create a simple and practical Student Management System

CHAPTER 3: CONCEPTS USED IN THE PROJECT

The project uses several important Python programming concepts to create a simple and functional application. The main concepts used are described below.

3.1 Variables

Variables are used to store and manage data in the program, such as student ID, student name, age, course, and marks.

3.2 Data Types

Data types tell Python what kind of value a variable store, such as strings for names and courses, integers for student ID and age, and floats for marks.

3.3 Functions

Functions are used to divide the program into smaller, reusable blocks of code. They make the program easier to understand, maintain, and reuse.

3.4 File Handling

File handling is used to create, read, write, and update files. It helps the application store student information permanently in students.txt.

3.5 OS Module

The OS module is used to interact with the operating system, for tasks such as checking whether a file exists, renaming files, and deleting files.

3.6 Date time Module

The Date time module is a built-in module used to work with dates and times, such as recording the date and time when a student record is added or updated.

3.7 Login System

A login system provides basic user authentication. Users enter a username and password before the main menu becomes available.

3.8 Activity Log

The activity log records important actions performed by the user, such as adding, updating, searching, and deleting student records, along with the date and time. This helps in tracking the usage of the application.

3.9 Conditional Statements

Conditional statements are used to make decisions in the program, such as checking whether login credentials are correct or whether a selected menu item exists.

3.10 Loops

Loops are used to repeat a block of code multiple times, such as displaying the menu, running the main menu until the user exits, or reading every student record from a file.

3.11 Duplicate Detection

Duplicate detection is the process of finding student records that are repeated. It can be implemented by checking whether a student ID already exists before adding a new student record.

CHAPTER 4: SOURCE CODE

The following is the complete, final Python source code for the Student Management System. It brings together all the operations used in the project, including student record management, file handling, login, searching, updating, and deleting student records.

```
import OS
```

```
from date time import date time
```

```
STUDENT_FILE = "students.txt"
```

```
LOG_FILE = "activity_log.txt"
```

```
# ----- LOGIN -----
```

```
def login():
```

```
    attempts = 0
```

```
    while attempts < 3:
```

```
        username = input ("Enter username: ")
```

```
        password = input ("Enter password: ")
```

```
        if username == "admin" and password == "123":
```

```
            log_ activity ("Login Success")
```

```
            print ("Login successful!")
```

```
            return True
```

```
        attempts += 1
```

```
        print (f "Invalid credentials. Attempts left: {3 - attempts}")
```

```
log_activity ("Login Failed")
```

```
print ("Too many failed attempts. Exiting.")
```

```
return False
```

```
# ----- ADD STUDENT -----
```

```
def add_student ():
```

```
    student_id = input("Enter Student ID: ")
```

```
    name = input("Enter Student Name: ")
```

```
    age = input("Enter Age: ")
```

```
    gender = input("Enter Gender: ")
```

```
    course = input("Enter Course: ")
```

```
    marks = input("Enter Marks: ")
```

```
if student_exists(student_id):
```

```
    print("Student ID already exists.")
```

```
    return
```

```
with open(STUDENT_FILE, "a", encoding="utf-8") as file:
```

```
    file.write(
```

```
        f"{student_id}|{name}|{age}|{gender}|{course}|{marks}\n"
```

```
    )
```

```
print("Student added successfully!")
```

```
log_activity(f"Student Added: {student_id}")
```

```
# ----- CHECK DUPLICATE -----
```

```
def student_exists(student_id):
```

```
    try:
```

```
        with open(STUDENT_FILE, "r", encoding="utf-8") as file:
```

```
            for line in file:
```

```
                data = line.strip().split("|")
```

```
                if data[0] == student_id:
```

```
                    return True
```

```
    except FileNotFoundError:
```

```
        return False
```

```
    return False
```

```
# ----- VIEW STUDENTS -----
```

```
def view_students():
```

```
    try:
```

```
        with open(STUDENT_FILE, "r", encoding="utf-8") as file:
```

```
            lines = file.readlines()
```

if not lines:

print("No student records found.")

return

print("\n----- STUDENT RECORDS -----")

for line in lines:

data = line.strip().split("|")

print(

f"ID: {data[0]}, "

f"Name: {data[1]}, "

f"Age: {data[2]}, "

f"Gender: {data[3]}, "

f"Course: {data[4]}, "

f"Marks: {data[5]]"

)

except FileNotFoundError:

print("No student file found.")

----- SEARCH STUDENT -----

def search_student():

search_id = input("Enter Student ID to search: ")

try:

with open(STUDENT_FILE, "r", encoding="utf-8") as file:

for line in file:

data = line.strip().split("|")

if data[0] == search_id:

print("\nStudent Found")

print("Student ID :", data[0])

print("Name :", data[1])

print("Age :", data[2])

print("Gender :", data[3])

print("Course :", data[4])

print("Marks :", data[5])

log_activity(f"Student Searched: {search_id}")

return

print("Student not found.")

except FileNotFoundError:

print("No student file found.")

----- UPDATE STUDENT -----


```
def update_student():  
    student_id = input("Enter Student ID to update: ")  
  
    try:  
        with open(STUDENT_FILE, "r", encoding="utf-8") as file:  
            lines = file.readlines()  
  
        found = False  
  
        with open(STUDENT_FILE, "w", encoding="utf-8") as file:  
  
            for line in lines:  
                data = line.strip().split("|")  
  
                if data[0] == student_id:  
                    found = True  
  
                print("Enter new student details:")  
  
                name = input("Enter Name: ")  
                age = input("Enter Age: ")  
                gender = input("Enter Gender: ")  
                course = input("Enter Course: ")  
                marks = input("Enter Marks: ")
```

```

        file.write(
            f"{student_id}|{name}|{age}|"
            f"{gender}|{course}|{marks}\n"
        )
    else:
        file.write(line)

if found:
    print("Student updated successfully!")
    log_activity(f"Student Updated: {student_id}")
else:
    print("Student not found.")

except FileNotFoundError:
    print("No student file found.")

# ----- DELETE STUDENT -----
def delete_student():
    student_id = input("Enter Student ID to delete: ")

    try:
        with open(STUDENT_FILE, "r", encoding="utf-8") as file:
            lines = file.readlines()

```

```
found = False
```

```
with open(STUDENT_FILE, "w", encoding="utf-8") as file:
```

```
    for line in lines:
```

```
        data = line.strip().split("|")
```

```
        if data[0] == student_id:
```

```
            found = True
```

```
        else:
```

```
            file.write(line)
```

```
if found:
```

```
    print("Student deleted successfully!")
```

```
    log_activity(f"Student Deleted: {student_id}")
```

```
else:
```

```
    print("Student not found.")
```

```
except FileNotFoundError:
```

```
    print("No student file found.")
```

```
# ----- COUNT STUDENTS -----
```

```
def count_students():
```

```
    try:
```

```
with open(STUDENT_FILE, "r", encoding="utf-8") as file:  
    count = len(file.readlines())
```

```
print("Total Students:", count)
```

```
except FileNotFoundError:  
    print("Total Students: 0")
```

```
# ----- ACTIVITY LOG -----
```

```
def log_activity(action):
```

```
    with open(LOG_FILE, "a", encoding="utf-8") as log:  
        log.write(  
            f"{datetime.now()} - {action}\n"  
        )
```

```
# ----- SHOW LOG -----
```

```
def show_log():
```

```
    try:  
        with open(LOG_FILE, "r", encoding="utf-8") as file:  
            print("\n----- ACTIVITY LOG -----")  
            print(file.read())
```

```
except FileNotFoundError:
```

```
print("No activity log found.")
```

```
# ----- MAIN MENU -----
```

```
def main():
```

```
    if not login():
```

```
        return
```

```
    while True:
```

```
        print("\n===== STUDENT MANAGEMENT SYSTEM  
=====")
```

```
        print("1. Add Student")
```

```
        print("2. View Students")
```

```
        print("3. Search Student")
```

```
        print("4. Update Student")
```

```
        print("5. Delete Student")
```

```
        print("6. Count Students")
```

```
        print("7. Show Activity Log")
```

```
        print("8. Exit")
```

```
        choice = input("Choose an option (1-8): ")
```

```
        if choice == "1":
```

```
            add_student()
```

elif choice == "2":

view_students ()

elif choice == "3":

search_student()

elif choice == "4":

update_student()

elif choice == "5":

delete_student ()

elif choice == "6":

count_students ()

elif choice == "7":

show_log()

elif choice == "8":

print ("Exiting program. Thank you!")

log_activity ("Program Exit")

break

else:

```
print ("Invalid choice. Please select 1-8.")
```

```
if __name__ == "__main__":  
    main ()
```

CHAPTER 5: WORKING EXPLANATION

5.1 Login and Setup

The program begins by calling `Login ()`, which asks the user for a username and password. The correct username is "admin" and the password is "123". The user has up to three attempts to enter the correct credentials. If all three attempts fail, the program exits.

Once logged in, the main menu of the Student Management System is displayed.

5.2 Add Student

- Gets the student's ID and name.
- Gets the student's age, gender, course, and marks.
- Checks whether the Student ID already exists.
- Saves the new student record in `students.txt`.

5.3 View Students

- Displays all available student records from the `students.txt` file.
- Reads the file and displays the Student ID, name, age, gender, course, and marks.

5.4 Search Student

The `search_student()` function asks the user to enter a Student ID. It searches the student file and displays the complete details of the matching student. If the ID is not found, it displays a "Student not found" message.

5.5 Update Student

The `update_student()` function searches for the given Student ID. If the student exists, the user can enter new details such as name, age, gender, course, and marks. The updated information is then saved back to the `students.txt` file.

5.6 Delete Student

The `delete_student()` function checks whether the given Student ID exists. If the student is found, the corresponding record is removed from the `students.txt` file. If the student does not exist, a "Student not found" message is displayed.

5.7 List

The `view_students()` function reads all student records from the `students.txt` file and displays each student with details such as Student ID, name, age, gender, course, and marks.

5.8 Search

The `search_student()` function asks the user to enter a Student ID and searches the student file for the matching record. If the student is found, the complete student details are displayed. Otherwise, a "Student not found" message is displayed.

5.9 Backup

The backup function creates a copy of the `students.txt` file with a timestamp. This helps to keep a backup of student records and prevents loss of important data.

5.10 Count

- The `len()` function is a built-in Python function used to find the number of elements in a list or the number of lines in a file.
- The `count_students()` function reads the `students.txt` file and displays the total number of student records.

5.11 History

The activity log stores the important actions performed in the Student Management System. It helps the user view previous activities such as adding, searching, updating, and deleting student records.

5.12 Extensions

- **.txt** → Plain text file used for storing student records.
- **.csv** → Comma-separated values file that can be used for structured student data.
- **PY** → Python source code file.

5.13 Duplicate Detection

The `student_exists()` function checks whether a Student ID already exists in the student file before adding a new record. This prevents the same student from being added more than once.

5.14 Activity Log

- The `log_activity()` function opens the `activity_log.txt` file in append mode ("a"), which adds new activity records without deleting previous entries.

- Every important action, such as login, adding a student, updating a record, searching, or deleting a student, is recorded with the date and time.

5.7 List

The `view_students()` function reads all student records from the `students.txt` file and displays each student with details such as Student ID, name, age, gender, course, and marks.

5.8 Search

The `search_student()` function asks the user to enter a Student ID and searches the student file for the matching record. If the student is found, the complete student details are displayed. Otherwise, a "Student not found" message is displayed.

5.9 Backup

The backup function creates a copy of the `students.txt` file with a timestamp. This helps to keep a backup of student records and prevents loss of important data.

5.10 Count

- The `len()` function is a built-in Python function used to find the number of elements in a list or the number of lines in a file.
- The `count_students()` function reads the `students.txt` file and displays the total number of student records.

5.11 History

The activity log stores the important actions performed in the Student Management System. It helps the user view previous activities such as adding, searching, updating, and deleting student records.

5.12 Extensions

- **.txt** → Plain text file used for storing student records.
- **.csv** → Comma-separated values file that can be used for structured student data.
- **.py** → Python source code file.

5.13 Duplicate Detection

The `student_exists()` function checks whether a Student ID already exists in the student file before adding a new record. This prevents the same student from being added more than once.

5.14 Activity Log

- The `Log Activity ()` function opens the `activity_log.txt` file in append mode ("a"), which adds new activity records without deleting previous entries.

- Every important action, such as login, adding a student, updating a record, searching, or deleting a student, is recorded with the date and time

- `log_activity ()` opens `activity_log.txt` in append mode ("a"), which adds new lines to the end of the file without erasing previous entries.

- Every time something important happens — a login, an order, a deletion — this function is called with a message describing the action.

- `"\n"` adds a new line so each log entry appears on its own row, along with a timestamp from `date time.now ()`.

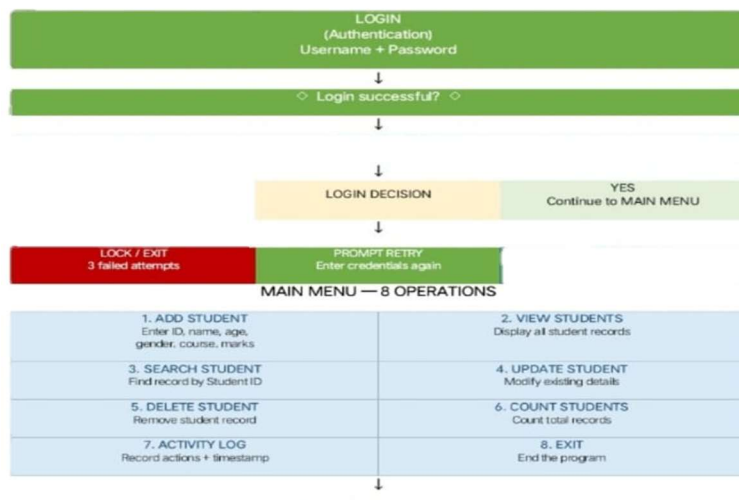
5.15 Flow Chart

The flow chart below shows the complete 15-step workflow of the application, from login (with a 3-attempt limit) through the 14 operations of the main menu to program exit.

5.15 FLOW CHART

STUDENT MANAGEMENT SYSTEM

Flowchart — Student Record Management Workflow



CHAPTER 6: OUTPUT

This chapter presents screenshots of the program running for every major operation, confirming that the source code in Chapter 4 behaves as described in Chapter 5. The login screenshots use the credentials username “Pooja” and password “123”, consistent with the source code and the flow chart.

6.1 SCREENSHOTS OF PROGRAM EXECUTION

6.1 LOGIN

```
> import ...

STUDENT_FILE = "students.txt"
LOG_FILE = "activity_log.txt"

# ----- LOGIN -----
def login(): 1 usage
    attempts = 0

    while attempts < 3:
        username = input("Enter username: ")
        password = input("Enter password: ")

        if username == "admin" and password == "123":
            log_activity("Login Success")
            print("Login successful!")
            return True
```

6.2 CREATE

```
# ----- ADD STUDENT -----
def add_student(): 1 usage
    student_id = input("Enter Student ID: ")
    name = input("Enter Student Name: ")
    age = input("Enter Age: ")
    gender = input("Enter Gender: ")
    course = input("Enter Course: ")
    marks = input("Enter Marks: ")

    if student_exists(student_id):
        print("Student ID already exists.")
        return

    with open(STUDENT_FILE, "a", encoding="utf-8") as file:
        file.write(
            f"{student_id}|{name}|{age}|{gender}|{course}|{marks}\n"
        )

    print("Student added successfully!")
    log_activity(f"Student Added: {student_id}")

# ----- CHECK DUPLICATE -----
def student_exists(student_id): 1 usage
    try:
        with open(STUDENT_FILE, "r", encoding="utf-8") as file:
            for line in file:
```

6.3 Read

```

# ----- VIEW STUDENTS -----
def view_students():
    1 usage
    try:
        with open(STUDENT_FILE, "r", encoding="utf-8") as file:
            lines = file.readlines()

            if not lines:
                print("No student records found.")
                return

            print("\n----- STUDENT RECORDS -----")

            for line in lines:
                data = line.strip().split("|")

                print(
                    f"ID: {data[0]}, "
                    f"Name: {data[1]}, "
                    f"Age: {data[2]}, "
                    f"Gender: {data[3]}, "
                    f"Course: {data[4]}, "
                    f"Marks: {data[5]}"
                )

    except FileNotFoundError:
        print("No student file found.")

```

6.4 Append

```

with open(STUDENT_FILE, "a", encoding="utf-8") as file:
    file.write(
        f"{student_id}|{name}|{age}|{gender}|{course}|{marks}\n"
    )

```

6.5 Rename

```
# ----- RENAME FILE -----  
def rename_file():  
    try:  
        new_name = input("Enter new file name: ")  
  
        os.rename(STUDENT_FILE, new_name)  
  
        print("File renamed successfully!")  
  
    except FileNotFoundError:  
        print("Student file not found.")
```

6.6 Delete

```

# ----- DELETE STUDENT -----
def delete_student(): 1 usage
    student_id = input("Enter Student ID to delete: ")

    try:
        with open(STUDENT_FILE, "r", encoding="utf-8") as file:
            lines = file.readlines()

        found = False

        with open(STUDENT_FILE, "w", encoding="utf-8") as file:

            for line in lines:
                data = line.strip().split("|")

                if data[0] == student_id:
                    found = True
                else:
                    file.write(line)

            if found:
                print("Student deleted successfully!")
                log_activity(f"Student Deleted: {student_id}")
            else:
                print("Student not found.")

    except FileNotFoundError:
        print("No student file found.")

```

6.7 List

```

# ----- VIEW STUDENTS -----
def view_students(): 1 usage
    try:
        with open(STUDENT_FILE, "r", encoding="utf-8") as file:
            lines = file.readlines()

            if not lines:
                print("No student records found.")
                return

            print("\n----- STUDENT RECORDS -----")

            for line in lines:
                data = line.strip().split("|")

                print(
                    f"ID: {data[0]}, "
                    f"Name: {data[1]}, "
                    f"Age: {data[2]}, "
                    f"Gender: {data[3]}, "
                    f"Course: {data[4]}, "
                    f"Marks: {data[5]}"
                )

    except FileNotFoundError:
        print("No student file found.")

```

6.7 Search

```

# ----- SEARCH STUDENT -----
def search_student(): 1 usage
    search_id = input("Enter Student ID to search: ")

    try:
        with open(STUDENT_FILE, "r", encoding="utf-8") as file:
            for line in file:
                data = line.strip().split("|")

                if data[0] == search_id:
                    print("\nStudent Found")
                    print("Student ID :", data[0])
                    print("Name      :", data[1])
                    print("Age       :", data[2])
                    print("Gender    :", data[3])
                    print("Course    :", data[4])
                    print("Marks     :", data[5])

                    log_activity(f"Student Searched: {search_id}")
                    return

            print("Student not found.")

    except FileNotFoundError:
        print("No student file found.")

```

6.8 Backup


```
def backup_students():
    try:
        backup_file = "students_backup.txt"

        with open(STUDENT_FILE, "r", encoding="utf-8") as source:
            with open(backup_file, "w", encoding="utf-8") as backup:
                backup.write(source.read())

        print("Backup created successfully!")

    except FileNotFoundError:
        print("No student file found.")
```

6.9 Count

```
# ----- COUNT STUDENTS -----
def count_students(): 1 usage
    try:
        with open(STUDENT_FILE, "r", encoding="utf-8") as file:
            count = len(file.readlines())

        print("Total Students:", count)

    except FileNotFoundError:
        print("Total Students: 0")
```

6.10 History+ Log – Run

```
# ----- SHOW LOG -----
def show_log(): 1 usage
    try:
        with open(LOG_FILE, "r", encoding="utf-8") as file:
            print("\n----- ACTIVITY LOG -----")
            print(file.read())

    except FileNotFoundError:
        print("No activity log found.")
```

6.11 Extension

```
STUDENT_FILE = "students.txt"
LOG_FILE = "activity_log.txt"
```

6.12 Duplicate

```
# ----- CHECK DUPLICATE -----
def student_exists(student_id): 1 usage
    try:
        with open(STUDENT_FILE, "r", encoding="utf-8") as file:
            for line in file:
                data = line.strip().split("|")

                if data[0] == student_id:
                    return True

    except FileNotFoundError:
        return False

    return False
```

6.13 Activity Log

```
# ----- ACTIVITY LOG -----
def log_activity(action): 7 usages
    with open(LOG_FILE, "a", encoding="utf-8") as log:
        log.write(
            f"{datetime.now()} - {action}\n"
        )
```

CONCLUSION

The Student Management System, developed using Python programming and file handling, successfully demonstrates the practical implementation of basic programming concepts and file operations. The project provides a simple and efficient way to manage student records through operations such as adding, viewing, searching, updating, deleting, and counting student details.

The system uses Python functions, conditional statements, loops, file handling, the OS module, and the date time module. A login system with limited attempts provides basic authentication, while duplicate student ID detection helps prevent duplicate records. The activity log records important actions performed in the system with date and time, making the application more organized and traceable.

This project improves the understanding of Python programming, file handling, data management, and modular programming. The use of separate functions for different operations makes the program easy to understand, maintain, and modify.

In the future, the Student Management System can be enhanced by adding a graphical user interface using Tkinter connecting the system to a database such as MySQL or SQLite, adding student attendance and grade management, and providing advanced search and reporting features. Overall, this project provides a useful and practical application for managing student information using Python.

BIBLIOGRAPHY / REFERENCES

For the successful completion of this mini project, the following sources were referred to:

Python Official Documentation – docs.python.org/3/tutorial/inputoutput.html – referred for file handling modes (w, r, a).

W3Schools – Python File Handling – [w3schools.com/python/python_file_handling.asp](https://www.w3schools.com/python/python_file_handling.asp) – for open (), read (), readlines (), and enumerate().

GeeksforGeeks – Python OS Module – [geeksforgeeks.org/os-module-python](https://www.geeksforgeeks.org/os-module-python) – for os.rename(), os.remove(), and os.path.exists().

Programiz – Python Lambda, Map, Filter – [programiz.com/python-programming](https://www.programiz.com/python-programming) – for lambda, map, filter, and generators.

Textbook References – “Python Programming” by Dr. R. Nageswara Rao (File Handling chapter); “Let Us Python” by Yashavant Kanetkar (Functions and File Management).

YouTube Tutorials – CodeWithHarry’s Python File Handling playlist; Telusko’s Python Project Tutorials.

Class Notes – Gowthami Mam’s lecture notes on file handling and mini project guidelines.